ASSIGNMENT-10.3

2303A51546

Batch-10

TASK-1

PROMPT:

Analyze the following Python code that calculates factorial.
Identify the logical error in the code and explain why it occurs.
Provide a corrected version and handle edge cases like negative numbers and zero.

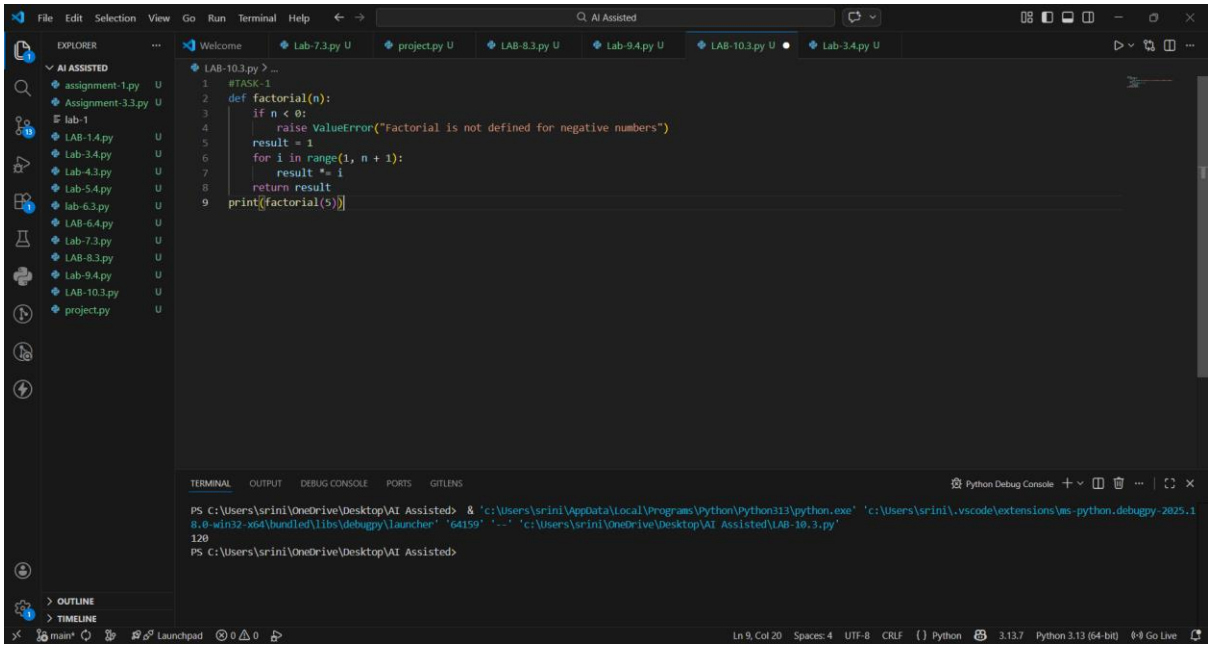
Original Code:(Buggy)

```
def factorial(n):
result = 1
for i in range(1, n):
result = result * i
return result
```

CODE:

```
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers")
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
print(factorial(5))
```

OUTPUT:



Analysis:

The function is written to calculate the factorial of a number.

Factorial means multiplying all numbers from 1 to n.

Example:

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

When we test the function using:

TASK-2

PROMPT:

Analyze the following Python function that performs arithmetic operations on two numbers. Review the readability, parameter naming, documentation, and error handling. Explain how the function works and how it handles invalid inputs.

Original code:

```
def calc(a, b, c):
if c == "add":
return a + b
elif c == "sub":
return a - b
```

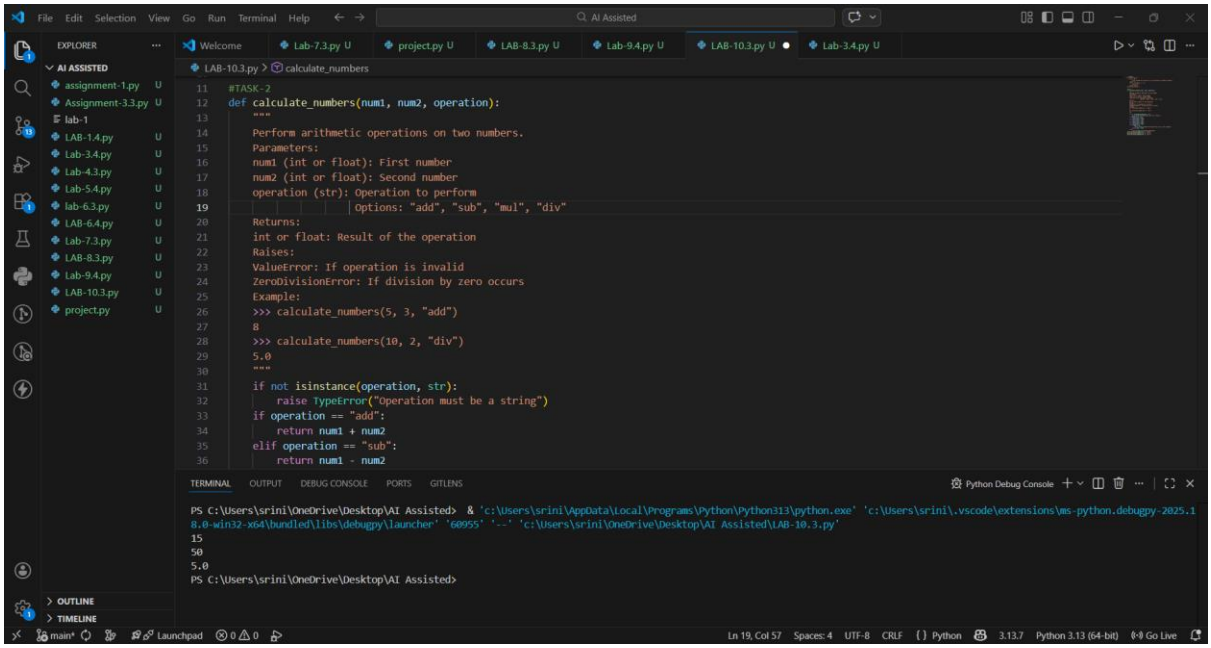
```
elif c == "mul":
return a * b
elif c == "div":
```

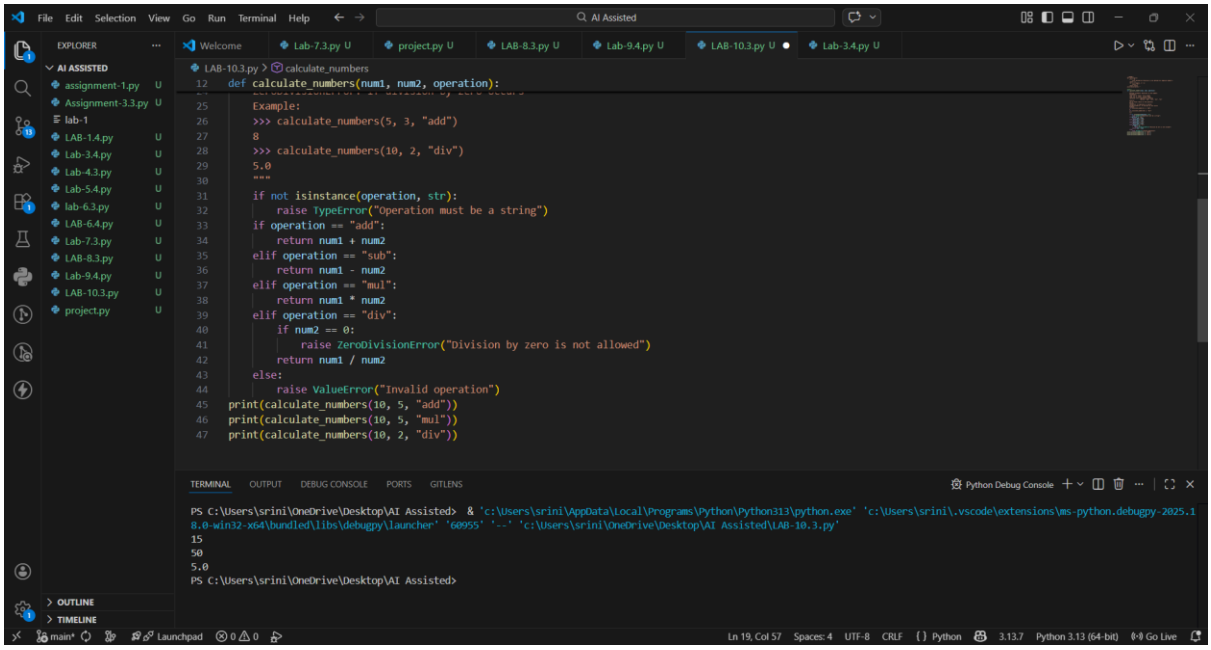
CODE:

```
def calculate_numbers(num1, num2, operation):
    """
    Perform arithmetic operations on two numbers.
    Parameters:
    num1 (int or float): First number
    num2 (int or float): Second number
    operation (str): Operation to perform
                Options: "add", "sub", "mul", "div"
    Returns:
    int or float: Result of the operation
    Raises:
    ValueError: If operation is invalid
    ZeroDivisionError: If division by zero occurs
    Example:
    >>> calculate_numbers(5, 3, "add")
    8
    >>> calculate_numbers(10, 2, "div")
    5.0
    """
    if not isinstance(operation, str):
        raise TypeError("Operation must be a string")
    if operation == "add":
```

```
        return num1 + num2
    elif operation == "sub":
        return num1 - num2
    elif operation == "mul":
        return num1 * num2
    elif operation == "div":
        if num2 == 0:
            raise ZeroDivisionError("Division by zero is not allowed")
        return num1 / num2
    else:
        raise ValueError("Invalid operation")
print(calculate_numbers(10, 5, "add"))
print(calculate_numbers(10, 5, "mul"))
print(calculate_numbers(10, 2, "div"))
```

OUTPUT:





Analysis:

The function calculate_numbers is used to perform arithmetic operations.

It takes three inputs: num1, num2, and operation.

num1 and num2 are numbers, and operation tells which operation to perform.

TASK-3:

PROMPT:

Analyze the following Python function that checks whether a number is prime. Explain how the function works and how it determines if a number is prime or not.

Original code:

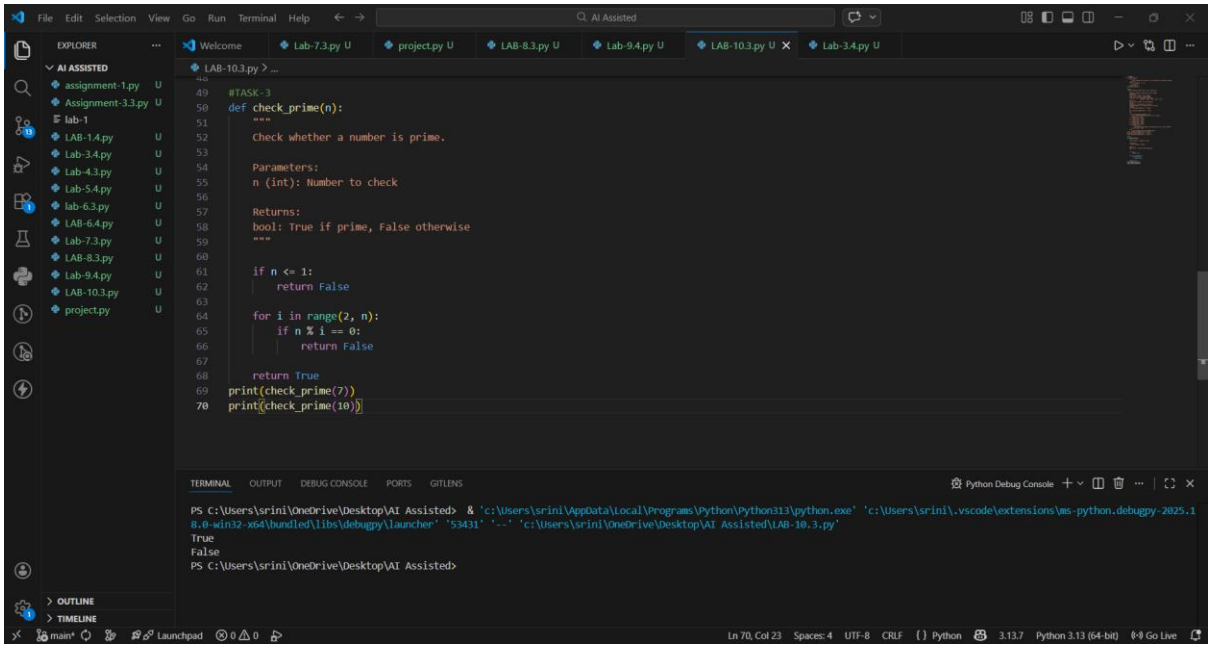
```
def Checkprime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
```

CODE:

```
def check_prime(n):
    """
    Check whether a number is prime.

    Parameters:
    n (int): Number to check

    Returns:
    bool: True if prime, False otherwise
    """

    if n <= 1:
        return False

    for i in range(2, n):
```

```
    if n % i == 0:
        return False


    return True
print(check_prime(7))
print(check_prime(10))
```

OUTPUT:



Analysis:

The function check_prime(n) is used to check whether a number is prime or not.

First, it checks if n ≤ 1. If it is, the function returns False because numbers less than or equal to 1 are not prime.

Then it uses a loop from 2 to n-1 to check if the number is divisible by any other number.

If n % i == 0, it means the number is divisible by another number, so the function returns False.

- If no number divides n, the function returns True, which means the number is prime.

TASK-4

PROMPT:

Analyze the following Python function that doubles the even numbers in a list. Explain how the function works and how it handles invalid input.

Original code:

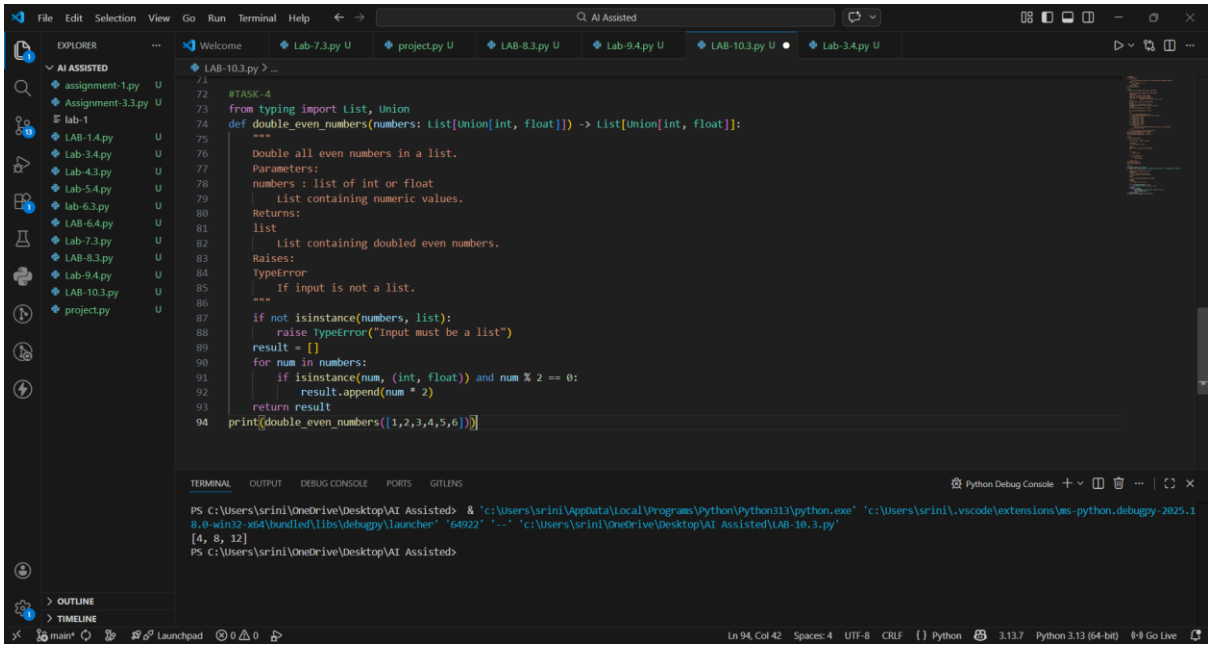
```
def processData(d):
return [x * 2 for x in d if x % 2 == 0]
```

CODE:

```
from typing import List, Union


def double_even_numbers(numbers: List[Union[int, float]]) ->
List[Union[int, float]]:
    """
    Double all even numbers in a list.


    Parameters:
    numbers : list of int or float
        List containing numeric values.


    Returns:
    list
        List containing doubled even numbers.
```

```
    Raises:
    TypeError
        If input is not a list.
    """

    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")

    result = []

    for num in numbers:
        if isinstance(num, (int, float)) and num % 2 == 0:
            result.append(num * 2)

    return result
print(double_even_numbers([1,2,3,4,5,6]))
```

OUTPUT:



Analysis:

The function double_even_numbers is used to double the even numbers in a list.

It first checks if the input is a list. If not, it shows an error.

Then it checks each number in the list using a loop.

If the number is even, it multiplies the number by 2.

The doubled numbers are stored in a new list and returned as output.

TASK-5

PROMPT:

Analyze the Python error shown below. Explain why the error occurs and how it can be fixed.

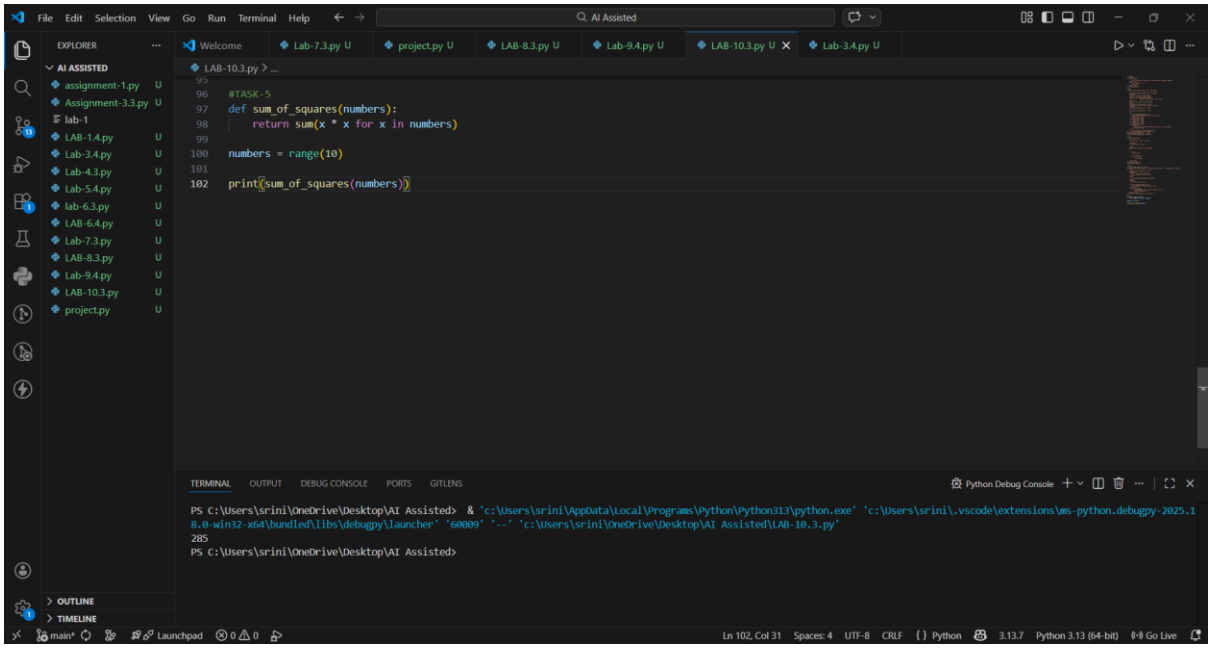
Original code:

```
def sum_of_squares(numbers):
total = 0
for num in numbers:
total += num ** 2
return total
```

CODE:

```
def sum_of_squares(numbers):
    return sum(x * x for x in numbers)


numbers = range(10)


print(sum_of_squares(numbers))
```

OUTPUT:



Analysis:

The program shows a NameError because Python cannot find the function sum_of_squares.

The code is trying to use the function before it is defined.

Python runs code from top to bottom, so the function must be written first.

To fix the error, define the sum_of_squares function before calling it.